

8 transformations Power Query qui cassent le refresh · en prod

LE RÉFLEXE ANTI-CASSE, EN 3 PAGES

Ça marche sur ton poste, ça plante au refresh planifié. Les 8 transformations fragiles, et **comment les blinder avant la mise en prod**.

Chaque module se lit seul, toujours dans le même ordre : le symptôme que tu observes, pourquoi ça casse précisément au rafraîchissement, le réflexe correctif à appliquer.

[01 · Nommage & typage](#)[02 · Dépendances de structure](#)[03 · Source & performance](#)[★ = le réflexe vital](#)

01 NOMMAGE & TYPAGE CODÉS EN DUR

les pièges 1 à 2

1.1 COLONNES RENOMMÉES EN DUR

SYMPTÔME

Le refresh échoue sur Table. RenameColumns ou un [Nom colonne] plus loin : « la colonne X du tableau n'a pas été trouvée ».

POURQUOI ÇA CASSE AU REFRESH

Le renommage référence le nom **source exact**. Le jour où la source renomme l'en-tête, ajoute un espace ou change la casse, l'étape ne retrouve plus sa cible et tout l'aval tombe.

RÉFLEXE CORRECTIF

★ **Renommer par position** quand l'ordre est garanti, ou ajouter MissingField. Ignore pour ne pas faire planter tout le flux sur un en-tête manquant.

Mieux : corriger les noms à la **source** (vue SQL, en-têtes stables) plutôt que dans Power Query.

1.2 TYPES IMPLICITES OU DEVINÉS

SYMPTÔME

Le Changed Type auto inséré à l'import passe en local, mais le refresh remonte des erreurs de type ou des cellules vides inattendues.

POURQUOI ÇA CASSE AU REFRESH

Le type a été **déduit d'un échantillon**. Une nouvelle valeur, une virgule décimale selon la locale, un texte dans une colonne numérique, et la conversion échoue sur des lignes qui n'existaient pas le jour de la conception.

RÉFLEXE CORRECTIF

★ **Typier chaque colonne explicitement** et en connaissance du contenu, pas en acceptant le type deviné.

Forcer la locale sur les conversions sensibles : type number et dates avec using "fr-FR" pour figer le séparateur décimal.

02 DÉPENDANCES DE STRUCTURE

les pièges 3 à 6

2.1 ÉTAPES DÉPENDANTES DE L'ORDRE DES COLONNES

SYMPTÔME

Un Table. SelectColumns par index, un {0}, un « supprimer les autres colonnes » qui fige une position : après refresh, les bonnes données sont dans la mauvaise colonne.

POURQUOI ÇA CASSE AU REFRESH

La transformation suppose un **ordre de colonnes** que la source ne garantit pas. Une colonne insérée ou réordonnée en amont décale tout, sans erreur visible : les valeurs deviennent silencieusement fausses.

RÉFLEXE CORRECTIF

★ **Référencer les colonnes par leur nom**, jamais par leur position ou leur index.

Sélectionner explicitement les colonnes voulues plutôt que « supprimer les autres ».

2.2 FUSION SUR CLÉS NON STABLES

SYMPTÔME

Un Table. NestedJoin qui marche, puis après refresh : lignes dupliquées, lignes perdues, ou jointure vide.

POURQUOI ÇA CASSE AU REFRESH

La clé n'est pas vraiment unique ou stable : un libellé au lieu d'un identifiant, une casse différente, un espace en fin de chaîne, un type qui diffère entre les deux tables. Le moindre écart d'un côté et la correspondance saute.

RÉFLEXE CORRECTIF

★ **Joindre sur une clé technique** (identifiant), pas sur un texte d'affichage.

Normaliser des deux côtés avant la jointure : Text.Trim, Text.Lower, même type. Vérifier l'unicité côté dimension.

2.3 EXPANSION DE TABLES IMBRIQUÉES

SYMPTÔME

Un `Table.ExpandTableColumn` (expand après jointure, ou colonne issue d'une API) qui plante au refresh : champ introuvable ou colonnes manquantes.

POURQUOI ÇA CASSE AU REFRESH

L'expansion **fige la liste des champs** vue à la conception. Si la structure imbriquée change (champ renommé, supprimé, ou absent ce jour-là), l'étape cherche une colonne qui n'existe plus.

RÉFLEXE CORRECTIF

- ★ **N'étendre que les champs réellement utilisés**, et dériver la liste dynamiquement si la structure varie.

Renommer dans la foulée de l'expansion pour ne pas propager des suffixes .1 instables.

2.4 REQUÊTES RÉFÉRENCÉES EN CASCADE

SYMPTÔME

Une chaîne de requêtes qui se référencent ($A \rightarrow B \rightarrow C$) : refresh très lent, et chaque correction sur A se propage de façon imprévisible.

POURQUOI ÇA CASSE AU REFRESH

Une requête référencée n'est **pas mise en cache** : chaque requête fille relit la source de zéro. Quatre requêtes filles, c'est la source interrogée quatre fois, et le folding qui se brise en chemin.

RÉFLEXE CORRECTIF

- ★ **Garder une chaîne courte et lisible**, isoler une requête de base unique, et désactiver le chargement des requêtes intermédiaires.

Surveiller le **query folding** sur la dernière étape de chaque branche.

03 SOURCE & PERFORMANCE

les pièges 7 à 8

3.1 CHANGEMENT DE SOURCE CODÉ EN DUR

SYMPTÔME

Le chemin du fichier, le nom du serveur ou de la base sont écrits en dur dans `Source` : ça marche depuis ton poste, le refresh planifié échoue (chemin introuvable, accès refusé).

POURQUOI ÇA CASSE AU REFRESH

Le service ne voit pas ton C : \Users\... ni ton lecteur réseau mappé. Et le jour où la source migre (recette vers prod), il faut éditer la requête, pas un paramètre.

RÉFLEXE CORRECTIF

- ★ **Externaliser source et chemin en paramètres** (requêtes de paramètres), un seul endroit à changer pour basculer d'environnement.

Préférer un chemin **UNC** ou une source accessible par la passerelle, jamais une lettre de lecteur locale.

3.2 TABLE.BUFFER MAL PLACÉ

SYMPTÔME

Un `Table.Buffer` ajouté « pour accélérer » qui rend le refresh plus lent, voire fait exploser la mémoire sur une grosse table.

POURQUOI ÇA CASSE AU REFRESH

`Table.Buffer` matérialise la table en mémoire et **casse le query folding** en aval : tout ce qui suit s'exécute côté Power Query au lieu de la source. Placé trop tôt sur une grosse table, c'est l'inverse de l'effet voulu.

RÉFLEXE CORRECTIF

- ★ **Ne bufferiser qu'une petite table relue plusieurs fois**, et le plus tard possible dans la chaîne.

Vérifier après ajout que le gain est réel et que le folding amont n'est pas sacrifié inutilement.

3.3 LE TEST FINAL AVANT LA PROD

LE REFRESH TIENT QUAND

1. **Aucune étape ne référence un nom source, une position ou un chemin en dur** : tout passe par les noms de colonnes, des clés techniques et des paramètres.
2. **Les types sont explicites et la locale figée** : une nouvelle valeur ou une autre machine ne change rien.
3. **Le query folding tient jusqu'à la dernière étape foldable** : pas de `Buffer` ni d'expansion qui le brise sans raison.

→ Imprime ces 3 pages et relis la checklist avant chaque publication : le refresh qui casse à 6h du matin se prévient à la conception.